

Pseudo Code

```
struct Course
    String courseNumber
    String name
    Vector<String> prerequisites

    Course(String number, String name)
```

Vector Pseudo Code

```
Vector<Course> loadFile(String fileName)
    open file of fileName
    initialize empty Vector<Course> returnVector
    if file open
        while line is valid, read each line
            if line contains at least one comma //check for at least two inputs
                parse line into vector of substrings separated by commas
                if substrings[0] contains a space //validate course number format
                    continue //skip line
                create Course(courseNumber = substring[0], name = substring[1])
                if substrings size > 2 //add prerequisites
                    for i = 2; substrings size < i; ++i
                        if returnVector contains substrings[i].courseNumber
                            add substrings[i] to Course.prerequisites
                add Course to returnVector
        close file
    return returnVector

bool contains(Vector<Course> courses, String courseNumber)
    for each course in courses
        if course.courseNumber equals courseNumber
            return True
    return False
```

```

void printCourse(Vector<Course> courses, String courseNumber)
    for all courses
        if the course is the same as courseNumber
            print out the course information
        for each prerequisite of the course
            print the prerequisite course information

```

Sorted Print

```

int partition(Vector<Course> courses, int low, int high)
    set pivot to courses[high].courseNumber
    set i equal to low - 1
    for ii = low to high
        if courses[ii].courseNumber < pivot
            increment i
            swap courses[i] and courses[ii]
    swap courses[i+1] and courses[high]
    return i+1

```

```

void quickSort(Vector<Course> courses, int low, int high)
    if low < high
        set par equal to partition(courses, low, high)
        quickSort(courses, low, par-1)
        quickSort(courses, par+1, high)

```

```

void printSortedCourses(Vector<Course> courses)
    set courses equal to quickSort(courses, 0, courses.size - 1)
    for each course in courses
        print course.courseNumber

```

Hash Table Pseudo Code

```

struct Node
    Course course
    unsigned int key
    Node pointer next

    Node()
        next = null
        key = max int

```

```
Node(Course)
    Node()
    course = Course
```

```
Node(Course, unsigned int aKey)
    Node(Course)
    key = aKey
```

```
HashTable<Course> loadFile(String fileName)
    open file of fileName
    initialize empty HashTable<Course> returnTable
    if file open
        while line is valid, read each line
            if line contains at least one comma //check for at least two inputs
                parse line into vector of substrings separated by commas
                if substrings[0] contains a space //validate course number format
                    continue //skip line
                create Course(courseNumber = substring[0], name = substring[1])
                if substrings size > 2 //add prerequisites
                    for i = 2; substrings size < i; ++i
                        if returnTable contains substrings[i].courseNumber
                            add substrings[i] to Course.prerequisites
                add Course to returnTable
        close file
    return returnTable
```

```
bool contains(HashTable<Course> courses, String courseNumber)
    hash courseNumber
    if node at index of hash in courses matches courseNumber
        return True
    else
        loop through node chain at hash
            return True if node matches courseNumber
    return False
```

```
void printCourse(HashTable<Course> courses, String courseNumber)
    for all Nodes in courses
        if node key equals courseNumber hash
            print out the node's course information
            for each prerequisite of the node's course
```

print the prerequisite course information

Sorted Print

```
void printSortedCourses(HashTable<Course> courses)
    initialize a temp Vector<Course>
    for each Node in courses
        add Node.course to temp
        if Node.next is not null
            initialize Node pointer nextNode to Node.next
            while nextNode is not null
                add nextNode.course to temp
                set nextNode to nextNode.next
    quickSort(temp, 0, temp.size - 1)
    for each course in temp
        print course.courseNumber
```

Tree Pseudo Code

```
struct Node
    Course course
    Node pointer left
    Node pointer right

    Node()
        Left and right equal null

    Node(Course)
        Node()
        course = Course

Tree<Course> loadFile(String fileName)
    open file of fileName
    initialize empty Tree<Course> returnTable
    if file open
        while line is valid, read each line
            if line contains at least one comma //check for at least two inputs
                parse line into vector of substrings separated by commas
                if substrings[0] contains a space //validate course number format
                    continue //skip line
                create Course(courseNumber = substring[0], name = substring[1])
```

```

        if substrings size > 2 //add prerequisites
            for i = 2; substrings size < i; ++i
                if returnTable contains substrings[i].courseNumber
                    add substrings[i] to Course.prerequisites
            add Course to returnTable
    close file
    return returnTable

```

```

bool contains(Tree<Course> courses, String courseNumber)
    set current Node pointer to courses.root
    while current is not equal to null
        if current course number equals courseNumber
            return True
        if courseNumber < current course number
            set current to current.left
        else
            set current to current.right
    return False

```

```

void printCourse(Tree<Course> courses, String courseNumber)
    set current Node pointer to courses.root
    while current is not equal to null
        if current course number equals courseNumber
            print out current course information
            for each prerequisite of current course
                print the prerequisite course information
        if courseNumber < current course number
            set current to current.left
        else
            set current to current.right

```

Sorted Print

```

void inOrder(Node pointer node)
    if node is not null
        recursive inOrder(node.left)
        print node.course.courseNumber
        recursive inOrder(node.right)

```

```
void printSortedCourses(Tree<Course> courses)
    inOrder(courses.root)
```

Menu Pseudo Code

```
void runMenu()
    initialize data structure for courses
    while user input is not equal to 9
        display menu options
        gather user input
        if input equals 1
            set courses to output of loadFile(CourseInfoFileName)
        else if input equals 2
            courses.printSortedCourses()
        else if input equals 3
            courses.printCourses("CSCI101")
        else if input not equal to 9
            print "Unrecognized Input"
```

Evaluation

Vector

Load File

Code	Line Cost	Times Run
open file of fileName	1	1
initialize empty Vector<Course> returnVector	1	1
if file open	1	1
while line is valid, read each line	1	n
if line contains at least one comma //check for at least two inputs	1	n
parse line into vector of substrings separated by commas	1	n
if substrings[0] contains a space //validate course number format	1	n
continue //skip line	1	n
create Course(courseNumber = substring[0], name = substring[1])	1	n
if substrings size > 2 //add prerequisites	1	n
for i = 2; substrings size < i; ++i	1	n

if returnVector contains substrings[i].courseNumber	$3n+1$	n
add substrings[i] to Course.prerequisites	1	n
add Course to returnVector	1	n
close file	1	1
return returnVector	1	1
Total Cost		$10n+5+n(3n+1)$
Runtime		$O(n)$

Contains

Code	Line Cost	Times Run
for each course in courses	1	n
if course.courseNumber equals courseNumber	1	n
return True	1	n
return False	1	1
Total Cost		$3n+1$
Runtime		$O(n)$

Hash Table

Load File

Code	Line Cost	Times Run
open file of fileName	1	1
initialize empty HashTable<Course> returnTable	1	1

if file open	1	1
while line is valid, read each line	1	n
if line contains at least one comma	1	n
parse line into vector of substrings separated by commas	1	n
if substrings[0] contains a space	1	n
continue	1	n
create Course(courseNumber = substring[0], name = substring[1])	1	n
if substrings size > 2	1	n
for i = 2; substrings size < i; ++i	1	n
if returnTable contains substrings[i].courseNumber	2n+5	n
add substrings[i] to Course.prerequisites	1	n
add Course to returnTable	1	n
close file	1	1
return returnTable	1	1
Total Cost		10n+5+n(2n+5)
Runtime		O(n)

Contains

Code	Line Cost	Times Run
------	-----------	-----------

hash courseNumber	1	1
if node at index of hash in courses matches courseNumber	1	1
return True	1	1
else	1	1
loop through node chain at hash	1	n
return True if node matches courseNumber	1	n
return False	1	1
Total Cost		2n+5
Runtime		O(n)

Tree

Load File

Code	Line Cost	Times Run
open file of fileName	1	1
initialize empty Tree<Course> returnTable	1	1
if file open	1	1
while line is valid, read each line	1	n
if line contains at least one comma	1	n
parse line into vector of substrings separated by commas	1	n

if substrings[0] contains a space	1	n
continue	1	n
create Course(courseNumber = substring[0], name = substring[1])	1	n
if substrings size > 2	1	n
for i = 2; substrings size < i; ++i	1	n
if returnTable contains substrings[i].courseNumber	7n+2	n
add substrings[i] to Course.prerequisites	1	n
add Course to returnTable	1	n
close file	1	1
return returnTable	1	1
Total Cost		10n+5+n(7n+2)
Runtime		O(n)

Contains

Code	Line Cost	Times Run
set current Node pointer to courses.root	1	1
while current is not equal to null	1	n
if current course number equals courseNumber	1	n
return True	1	n

if courseNumber < current course number	1	n
set current to current.left	1	n
else	1	n
set current to current.right	1	n
return False	1	1
Total Cost		7n+2
Runtime		O(n)

Advantages/Disadvantages

Vector Advantages

- Fast access $O(1)$
- Easy to iterate through due to sequential memory storage
- Uses less space

Vector Disadvantages

- Slower search times
- More complex insertion/deletion
- Slow to resize

Hash Table Advantages

- Flexible size
- Quick search/insertion/deletion, $O(1)$ on average

Hash Table Disadvantages

- Needs collision handling
- More complex to iterate over

Tree Advantages

- Good average search/insertion/deletion, $O(\log(n))$
- Naturally sorted
- Flexible size

Tree Disadvantages

- Requires more memory due to dynamic elements
- Can become a skewed tree resulting in poor search/insertion/deletion performance

Recommendation

Based on this analysis I recommend a Hash Table. The hash table is a good middle ground between the dynamic nature of the binary search tree and the sequentially stored nature of the vector. It also does not have the balancing issue a BST has or the slow resizing issue a vector has. The hash table provides an average time complexity of $O(1)$ for insertions, and deletion, of elements and element searches due to its use of a hash key. The main downside is it may be a little more complex to maintain when element collisions happen, and a good hash key function will be important to establish. Overall the code may be more complex to set up but the best time complexities, $O(1)$, and the worst time complexities, $O(n)$, average out to be better or equal to the other options.

Vector has a best and worst of $O(1)$ for element access and a best and worst of $O(n)$ for search/insertion/deletion.

BST has a best of $O(\log(n))$ for access/search/insertion/deletion, and a worst of $O(n)$ for access/search/insertion/deletion.